

algorithm *iFFD*

Input : $Alloc_o, \mathbf{Used}$ **Output** = $Alloc_n$
 $Donors = \phi, Receivers = \phi$
For all servers S_j
 $Prev_j = \text{sum of VMs in } S_j \text{ by } Alloc_o$
 If ($Prev_j > Used_j$)
 Add S_j to *Donors*
 $Mig_j = Prev_j - Used_j$
 Else
 Add S_j to *Receivers*
 End - For
For all S_j in *Donors*
 Pick the smallest VMs that add upto
 Mig_j and add them to *MigList*
End - For
Sort *MigList* based on size
For all VM_i in *MigList*
 Place VM_i on the first *Donor* $_j$ that
 can pack it within $Used_j$
End - For
Return $Alloc_n$

(a)

algorithm *pMaP*

Input : $Alloc_o, VM_i$ **Output** : *Migs*
 $Alloc_n = mPPH(Alloc_o, VM_i)$
 $MList = \text{getMigList}(Alloc_o, Alloc_n)$
 $\forall Server_j$ with no VMs placed in $Alloc_n$
 $VG_j = \text{VMs placed on } Server_j \text{ in } Alloc_o$
 Add VG_j to *MList*
 $\forall mig_i \in MList$
 $Cost_i = \text{getMigrationCost}(mig_i)$,
 $Benefit_i = \text{getBenefit}(mig_i)$
Sort *MList* by $Benefit_i / cost_i$ (decreasing)
 $mig_{best} = \text{most profitable entry in } MList$
while ($profit_{best} > cost_{best}$) AND ($MList \neq \Phi$)
 $Migs = Migs \cup mig_{best}$
 Delete mig_{best} from *MList*
 Recompute Cost and Benefit for *MList*
End While
return *Migs*
end *pMaP* Algorithm

(b)

Fig. 6. (a) History Aware Packing Algorithm, (b) Migration Cost-aware Locally Optimal Placement Algorithm

the VM VM_i from S_j (at load ρ_j) to S_k (at load ρ_k). Also, since we had selected S_j over S_k for the additional load, there exists some VM $VM_{i'}$ for which the slope of S_j at load ρ_j is less than the slope for the server S_k at load ρ_k . However, by the *Ordering* assumption, such a VM $VM_{i'}$ can not exist. This leads to a contradiction and proves the required result.

The proof for the *Background Independence Property* is straightforward. If this property holds, then the incremental load due to application VM_i is independent of the workload mix, and as a result, we can always compute the exact increase in power on all candidate servers for VM_i and pick the best server. Hence, the shares allocated to the servers are locally optimal. This completes the proof.

One may also observe that if all the power models are concave, then the utilization allocation obtained is globally optimal as well. However, we did not observe this property to hold in our experiments and concluded that *mPP* can provably lead to locally optimal allocations only.

The *mPP* algorithm is designed to minimize power. However, it is oblivious of the last configuration and hence may entail large-scale migrations. This may lead to a high overall (power + migration) cost. Hence, we next propose a variant of *FFD* called *incremental FFD* (*iFFD*) for packing the applications on physical servers, given a fixed target utilization for each server in Fig. 6(a).

iFFD first computes the list of servers that require higher utilization in the new allocation, and labels them as receivers. For each donor (servers with a target utilization lower than the current utilization), it selects the smallest sized applications to migrate and adds them to a VM migration list. It then runs *FFD*